RAIDFLASH PD

Prefill and Decode occupancy. One job fills the die.

Jonathan Briceño De Luise · Peru, b. 1982 · October 2026 · v1.0

Abstract

Serving stacks already open N concurrent streams. Published ceilings are tokens at high concurrency (StorageReview 2×GB300 Flash: 353 tok/s at one stream, 2721 at 32). RAIDFLASH assigns those streams to one job — or, when several people are present, splits the filled die. Prefill (P) and decode (D) share the same lane pool and have different knees.

On 2×GB10 TensorFold GLM-5.3-Flash EXL3 TP2, live `usage` tokens with `delta==sum`:

• D: 49.5 → 121.1 tok/s at k=8 (2.45× vanilla C1).
• P-hot: drafted seed (`max_tokens ≥ 16`) then k sisters, ≥98.7% cache, 5.94× vs serial seeds; 8k document packer 94.6 tok/s, 2064 completion tokens in 21.8 s.
• P-cold: ~1600 tok/s already at k=1 (GEMM full). Eight distinct 8k prompts: 2317 → 8806 tok/s.

Vanilla on this box is one stream (~50 tok/s) while seven seats sit idle. RAIDFLASH 1.0 is occupancy, not a new kernel. Copy the law, not the 8.

```
k = min(PARALLEL, k_codo_phase)
```

1. Origin

RAIDFLASH is not a TensorFold fork and not a new attention kernel. It is a policy for who the engine's concurrent paths belong to.

Commercial serving keeps a GPU busy with N tenants so each cursor stays “interactive” (~50 tok/s) while the die does tens of thousands. A clinic, a lab, a home box bought the silicon for the person in front of it. Those N paths can be stripes of one document. When more people show up, the same map splits maximum compute instead of rationing C1.

The idea came from outside the serving lab: a person looking at an idle GPU while a human waited.

Author: Jonathan Briceño De Luise, born in Peru, 1982.

2. Law (P and D)

`PARALLEL` is the seat cap. `k_codo` is the last k that still raises aggregate tok/s. Idle seats below the knee are a miss. Seats past the knee are a miss.

phase	bound	k on this GB10
P-cold	compute / GEMM	1 per document (extra k does not raise agg)
P-hot	KV after a drafted seed	8 sisters, same prefix, ~99% cache

phase	bound	k on this GB10
D	memory / expert reload	8 (2.45× C1; still pays)

Draft depth (`DFlash most=7`) is depth of one chain, not lane count.
At k=8, `MULTI_WINDOW=32` leaves 4 verify rows/stream so most fits as 4.
Short thinking (`max_tokens ≤ 80`) keeps k=4 so most=7 still fits.

Injector on 8 seats:

people	D / P-hot	P-cold
1	`[8]`	`[1]`
3	`[3, 3, 2]`	`[1, 1, 1]`

A B300 map may want 32 / 64 / 120. Measure. Do not copy GB10's 8 onto HBM.

3. How a job is cut

1. Seed (P). Prefill the document once. `max_tokens ≥ 16` so DFlash keep is resumable. Seed mt=8 → 0% sister cache (measured).
2. Sisters (P-hot + D). k streams: same prefix + “section i”. Prefix hits cache. Decode of the k stripes runs together.
3. Merge. One `POST /v1/raidflash/complete`. The client sees one completion. The GPU saw k streams.

P-hot is KV, not speculation. The GEMM either runs or the prefix is already there.

4. This engine (EFI, `delta==sum`)

2× NVIDIA GB10 Spark, GLM-5.3-Flash EXL3, TensorFold CUDA, `--parallel 8`, `PREFILL_ROWS=4096`, `FILL_ROWS=2048`, fat pool ~2.03M tokens, tree OFF.
Live image `raidflash-glm53:1.0.4`, `RAIDFLASH_PREFIX=0`.

Decode (EFI shorts)

k	agg tok/s	vs C1	wall for that cell
1	49.5	1.00×	1.62 s
2	71.0	1.43×	2.25 s
4	81.1	1.64×	3.95 s
8	121.1	2.45×	5.28 s

Prefill cold

k	agg prefill tok/s
1	1586
8	1629 (flat)

Eight independent ~8k prompts: 2317 → 8806 tok/s.

Prefill hot (seed + sisters, mt=16)

k	cache min	vs serial seeds
8	99.0%	5.94× (wall 5.40 s vs ~32 s serial)

Packer 8k document, stripe 256 (Engine 1.0.4)

completion	2064 (seed 16 + 8×256)
cache	98.7%
wall	21.8 s
agg	94.6 tok/s
sister prefill	0.83 s after seed 3.90 s

Sister `completion_tokens` are generated tokens, not DFlash `accepted`.
Counting `accepted` under-reported ~150 tok/lane (agg 62). Fixed in 1.0.4.

5. Seconds and tokens versus vanilla C1

Vanilla = one stream on the same silicon. A short Hermes chat stays 1 lane (~50 tok/s). Occupancy pays when the job fills seats.

job	vanilla C1	RAIDFLASH PD	gain
1024 prose	30.2 s · 33.9 tok/s	12.8 s · 80.3 tok/s (8×128)	−17.4 s
2048 D	~37 s @ 55 tok/s	16.9 s @ 121 tok/s	−20 s
5000 prose	44 s, stops at 1590 tok	56.5 s, 5000 tok (8×625)	+3410 tok
8k document, 8×256	~45 s serial	21.8 s · 94.6 tok/s	−23 s
8 cold 8k prefs	~2317 tok/s	8806 tok/s	~3.8× prompt
code 1024	57 tok/s	109 tok/s	+52
thinking 256	45 tok/s	115 tok/s	+70

Same-budget 1024: occupancy would emit ~2400 tokens in vanilla's 30 s.

6. Product 1.0

In-engine occupancy: `POST /v1/raidflash/complete`, every chat is a `ChatJob` (`job_id`), EFI table per machine (`raidflash probe`), `raidflash:*` metrics, job-first queue. Surgical TensorFold overlay (`DepthPolicy.most`, never clobber `fnc` with `fc7`). Flag off = stock engine.

Recipe that paid: `--parallel 8`, fat KV pool, `MULTI_WINDOW=32`, seed ≥16, RAIDFLASH-D most=7 unfixed.

Falsified here (leave off): tree T3, request-level P‖D (−59% D), `MULTI_WINDOW=64`, k=16 as default (D peak dropped), seed mt=8 (0% cache), prefix-kqi with too much SMEM on GB10 (D 107→83; lab only).

MLA / Hydragen is Dynamo, out of this stack. Neighbour: IPPD (arXiv 2609.05707).

CAKE is the product that answers people. RAIDFLASH is the occupancy layer so that answer spends the chip they paid for.

7. Close

v1.0 methods are crude on purpose: a seed, k sisters, an EFI table, one POST. The community will write sharper kernels. The disruption is the permission:

Run the GPU you already own at 100%.

People who were about to buy another card can use the one they paid for.

Sovereign AI. Local AI. CAKE.

Tables: TABLE.md. Law: OCCUPANCY.md. Map: [EFI.md](EFI.md).

Apache-2.0. Overlay only — TensorFold is not vendored.